
CP Reference Book

Python Edition

Adán Arteaga

A contest-ready reference for competitive programming
and technical interviews

Every snippet runs as written — no pseudocode, no placeholders.
Complexities are stated where the choice matters.

Landscape layout, two columns, built for printing.
Clickable table of contents and full PDF bookmarks for on-screen use.

Contents

1	Contest Setup and Complexity	1
2	Python Toolkit	2
3	Math, Modular Arithmetic and Number Theory	4
4	Arrays, Prefix Sums and Searching	7
5	Bit Manipulation	10
6	Strings	11
7	Graphs: Traversal and Ordering	13
8	Shortest Paths, MST and DSU	15
9	Trees and Advanced Graphs	17
10	Dynamic Programming	18
11	Advanced Data Structures	20
12	Contest Techniques and Pitfalls	22

1 Contest Setup and Complexity

■ Full Template

```
import sys
from math import gcd, isqrt, inf
from collections import deque, defaultdict, Counter
from bisect import bisect_left, bisect_right
from heapq import heappush, heappop
from itertools import accumulate, permutations, combinations

input = sys.stdin.readline

INF = float('inf')
MOD = 10**9 + 7

def solve():
    n = int(input())
    a = list(map(int, input().split()))
    # ...

def main():
    t = int(input())      # remove if single test case
    for _ in range(t):
        solve()

main()
```

Tip: Wrapping the work in a function is a real speedup: CPython resolves local variables by index and globals by dictionary lookup. The same loop runs noticeably faster inside `main()` than at module level.

■ Fast Input

```
import sys
input = sys.stdin.readline

n = int(input())
a = list(map(int, input().split()))
```

Careful: `sys.stdin.readline` keeps the trailing newline. For integers `int()` strips it, but for strings you must call `.strip()` or `.rstrip()` yourself.

■ Reading Everything at Once

```
import sys

data = sys.stdin.buffer.read().split()
it = iter(data)

n = int(next(it))
a = [int(next(it)) for _ in range(n)]
```

Use: the fastest input method in Python. With 10^6 numbers this is several times quicker than looping over `input()`.

■ Fast Output

```
import sys

out = []
for x in results:
    out.append(str(x))
sys.stdout.write("\n".join(out) + "\n")
```

Careful: Calling `print()` 10^5 times is slow enough to TLE on its own. Buffer the lines and write once.

■ Recursion Limit

```
import sys
sys.setrecursionlimit(300000)
```

Careful: Raising the limit does not enlarge the C stack — deep recursion still segfaults. For $n \geq 10^5$, write the DFS iteratively.

■ Deep Recursion via a Thread

```
import sys, threading

def main():
    # recursive solution here
    ...

sys.setrecursionlimit(1 << 25)
threading.stack_size(1 << 27)      # 128 MB stack
threading.Thread(target=main).start()
```

Use: when a recursive solution is far cleaner than the iterative one. A thread gets its own stack, whose size you control.

■ Constants

```
INF = float('inf')      # compares correctly, but is a float
BIG = 10**18            # use when you need a real integer
MOD = 10**9 + 7
MOD2 = 998244353
```

Careful: `float('inf')` used in integer arithmetic silently turns results into floats and loses exactness past 2^{53} . In DP tables prefer `10**18`.

■ Complexity Classes

- $O(1)$ — index access, average dict/set lookup.
- $O(\log n)$ — binary search, heap push/pop.

- $O(\sqrt{n})$ — divisors, simple primality.
- $O(n)$ — a single pass.
- $O(n \log n)$ — sorting, most greedy solutions.
- $O(n^2)$ — nested loops, many DP tables.
- $O(2^n)$ — subsets, bitmask DP.
- $O(n!)$ — permutations.

■ Constraint to Complexity

Python runs roughly 10^7 interpreted operations per second — about an order of magnitude slower than C++.

Constraint	Target	Python
$n \leq 10$	$O(n!)$	fine
$n \leq 20$	$O(2^n n)$	fine
$n \leq 500$	$O(n^3)$	risky
$n \leq 5000$	$O(n^2)$	risky
$n \leq 10^5$	$O(n \log n)$	fine
$n \leq 10^6$	$O(n)$	keep the loop simple
$n \leq 10^{18}$	$O(\log n)$	fine

Tip: When only the constant factor is the problem, push the loop into C: `sum()`, `map()`, `sorted()`, slicing and `itertools` all run at native speed. A comprehension beats an explicit `for` with `.append()`.

2 Python Toolkit

■ List Essentials

```
a = [0] * n          # n zeros
g = [[0] * m for _ in range(n)]  # n x m grid

a.sort()
a.sort(reverse=True)
b = sorted(a)        # returns a new list
a.reverse()

mn, mx = min(a), max(a)
s = sum(a)
c = a.count(x)
i = a.index(x)       # ValueError if absent
```

Careful: `[[0]*m]*n` creates n references to the *same* row. Writing `g[0][0]` changes every row. Always use the comprehension form.

■ Sorting with Keys

```
pts.sort()              # lexicographic
pts.sort(key=lambda p: p[1])  # by second field
```

■ Cost of Python Structures

Operation	Cost
<code>list</code> append / pop back	$O(1)$ amortized
<code>list</code> insert / pop(0)	$O(n)$
<code>list</code> in	$O(n)$
<code>dict</code> / <code>set</code> get, add	$O(1)$ average
<code>set</code> in	$O(1)$ average
<code>deque</code> push/pop both ends	$O(1)$
<code>heapq</code> push / pop	$O(\log n)$
<code>bisect</code> search	$O(\log n)$
<code>list.insert</code> after <code>bisect</code>	$O(n)$
<code>sorted()</code> / <code>.sort()</code>	$O(n \log n)$
string concat in a loop	$O(n^2)$

Careful: Two silent quadratics: `a.pop(0)` in a loop (use `deque`) and `s += ch` in a loop (collect into a list and `"".join()`).

■ PyPy vs CPython

- Choose PyPy when it is offered — typically 5–30× faster on tight loops.
- PyPy dislikes very large ints; CPython handles them well.
- On PyPy, plain loops are fast; on CPython, push work into built-ins.
- Recursion is slower on PyPy than on CPython. Prefer iterative.

```
pts.sort(key=lambda p: (p[0], -p[1]))  # asc, then desc
words.sort(key=len)

# Sort indices instead of the data
order = sorted(range(n), key=lambda i: a[i])
```

Tip: Negating a key gives a descending sub-order without a custom comparator. For non-numeric keys, use `functools.cmp_to_key`.

■ Slicing

```
a[l:r]      # elements l .. r-1 (r excluded)
a[:k]       # first k
a[-k:]      # last k
a[::-1]     # reversed copy
a[::2]      # every other element
a[l:r] = [] # delete a slice in place
```

Careful: A slice is a *copy*. Slicing inside a loop turns $O(n)$ into $O(n^2)$ — use indices instead.

■ Dicts, Sets and Counters

```

from collections import defaultdict, Counter

freq = defaultdict(int)
for x in a:
    freq[x] += 1

cnt = Counter(a)
cnt.most_common(3)      # 3 most frequent pairs
cnt['missing']          # 0, and does NOT insert

adj = defaultdict(list)
adj[u].append(v)

seen = set()
seen.add(x)
seen.discard(x)         # no error when absent

a_set & b_set            # intersection
a_set | b_set           # union
a_set - b_set           # difference
a_set ^ b_set           # symmetric difference

```

Careful: Reading `d[k]` on a `defaultdict` creates the key. Inside a loop that quietly grows the dict — use `d.get(k, 0)` for read-only access.

■ deque

```

from collections import deque

dq = deque()
dq.append(x)      ; dq.appendleft(x)
dq.pop()          ; dq.popleft()
dq[0]            ; dq[-1]
dq = deque(maxlen=k) # auto-evicts from the left

```

Use: BFS queues, sliding window extrema, 0–1 BFS.

■ heapq

```

from heapq import heappush, heappop, heapify, nlargest

pq = []
heappush(pq, (dist, node))
d, u = heappop(pq)      # ALWAYS the smallest
heapify(a)              # O(n), in place

# Max-heap: negate on the way in and out
heappush(pq, -x)
x = -heappop(pq)

top3 = nlargest(3, a)

```

Careful: `heapq` is min-only. There is no `heappop_max` in the public API.

Tip: Tuples compare element by element, so `(cost, node)` sorts by cost automatically — which is exactly what Dijkstra needs.

■ math Module

```

from math import (gcd, lcm, isqrt, comb, perm,
                 factorial, log2, ceil, floor, inf)

gcd(a, b)          # any number of args in 3.9+
lcm(a, b)          # 3.9+
isqrt(n)           # EXACT integer sqrt
comb(n, r)         # exact nCr, arbitrary precision
perm(n, r)         # exact nPr, arbitrary precision
pow(a, b, MOD)     # fast modular exponentiation, built in

```

Careful: `int(n ** 0.5)` is wrong for large `n`: floating point rounds and you get an off-by-one. Always use `isqrt`.

■ Integer Division and Modulo

```

7 // 2      # 3
-7 // 2     # -4  floor division, NOT truncation
-7 % 3      # 2   the result always has the sign of the divisor
divmod(7, 2) # (3, 1)

# C-style truncation toward zero
int(-7 / 2) # -3

```

Use: `x % MOD` is already non-negative in Python for positive `MOD`, so the `(x % M + M) % M` dance from C++ is unnecessary.

■ Big Integers

```

x = 2 ** 1000      # exact, no overflow, ever
factorial(100)     # all 158 digits

```

Use: no overflow bugs. Python removes the single most common C++ error class for free.

Careful: Arithmetic on huge integers is not $O(1)$ — it scales with digit count. Keep taking `% MOD` inside loops.

■ itertools

```

from itertools import (accumulate, permutations, combinations,
                      product, groupby, chain)

list(accumulate([1,2,3]))      # [1, 3, 6] prefix sums
list(accumulate(a, max))       # running maximum
permutations(a, 2)             # all ordered pairs
combinations(a, 2)             # unordered pairs
product(range(2), repeat=n)    # all n-bit tuples
list(chain(*lists))            # flatten one level

# Run-length encoding
for ch, grp in groupby(s):
    print(ch, len(list(grp)))

```

■ Memoization

```

from functools import lru_cache, cache

```

```
@cache                    # 3.9+, unbounded
def f(i, j):
    if i == 0:
        return 0
    return min(f(i - 1, j), f(i, j - 1)) + cost[i][j]

@lru_cache(maxsize=None)  # equivalent, any version
def g(state):
    ...

f.cache_clear()          # between test cases!
```

Careful: Arguments must be hashable — pass tuples, never lists. And clear the cache between test cases, or answers leak from one to the next.

Tip: Top-down memoization is often the fastest way to a correct DP under contest pressure. Convert to bottom-up only if it TLEs.

Custom Sort Comparator

```
from functools import cmp_to_key
```

```
# Order strings so their concatenation is largest
def cmp(a, b):
    if a + b > b + a: return -1
    if a + b < b + a: return 1
    return 0

arr.sort(key=cmp_to_key(cmp))
```

Careful: `cmp_to_key` is noticeably slower than a plain key. Use a key function whenever the order can be expressed as one.

Memory-Lean Containers

```
from array import array

a = array('i', [0]) * n        # compact int32 array
flags = bytearray(n)          # 1 byte per boolean
grid = [input().rstrip() for _ in range(n)] # strings
```

Tip: A Python `list` of ints costs ≈ 8 bytes per pointer *plus* the object. `bytearray` and `array` cut memory by an order of magnitude when you are near the limit.

3 Math, Modular Arithmetic and Number Theory

GCD and LCM $\mathcal{O}(\log \min(a, b))$

```
from math import gcd, lcm    # lcm needs 3.9+

def lcm_safe(a, b):
    if a == 0 or b == 0:
        return 0
    return abs(a // gcd(a, b) * b)

def gcd_manual(a, b):
    while b:
        a, b = b, a % b
    return abs(a)
```

Properties:

- $\gcd(a, b) \cdot \text{lcm}(a, b) = |a \cdot b|$.
- $\gcd(a, b) = \gcd(b, a \bmod b)$.
- $\gcd(a, b, c) = \gcd(a, \gcd(b, c))$.

Tip: `gcd(*a)` folds the gcd over a whole list in one call.

Extended Euclid $\mathcal{O}(\log \min(a, b))$

Solves $ax + by = \gcd(a, b)$.

```
def extgcd(a, b):
    if b == 0:
        return a, 1, 0
```

```
g, x1, y1 = extgcd(b, a % b)
return g, y1, x1 - (a // b) * y1
```

Use: modular inverse for a non-prime modulus, and linear Diophantine equations.

Linear Diophantine

$ax + by = c$ is solvable iff $\gcd(a, b) \mid c$.

```
def solve_dio(a, b, c):
    g, x, y = extgcd(a, b)
    if c % g:
        return None
    k = c // g
    return x * k, y * k, g
# General family: x + t*(b/g), y - t*(a/g)
```

Modular Exponentiation $\mathcal{O}(\log b)$

```
ans = pow(a, b, MOD)        # built in, C speed

def binpow(a, b, mod):      # the manual version
    res = 1
    a %= mod
    while b:
        if b & 1:
            res = res * a % mod
        a = a * a % mod
        b >>= 1
```

```
return res
```

Tip: Always use the built-in three-argument `pow`. It is written in C and beats any Python loop.

■ Modular Inverse $\mathcal{O}(\log m)$

```
# MOD prime (Fermat)
def inv(a, mod=MOD):
    return pow(a, mod - 2, mod)

# Python 3.8+: works for ANY coprime modulus
def inv_any(a, mod):
    return pow(a, -1, mod)    # ValueError if not coprime
```

Careful: The three-argument `pow` with a negative exponent needs Python 3.8 or newer. On older judges fall back to `extgcd`.

■ Modulo Properties

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m)) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$$

Use: Python's `%` already returns a non-negative value for a positive modulus, so no correction term is needed.

Careful: Division still does not distribute. Multiply by the inverse.

■ Factorials and `nCr` $\mathcal{O}(n \text{ precompute, } 1 \text{ per query})$

```
MAXN = 10**6 + 5
fact = [1] * MAXN
for i in range(1, MAXN):
    fact[i] = fact[i - 1] * i % MOD

invfact = [1] * MAXN
invfact[MAXN - 1] = pow(fact[MAXN - 1], MOD - 2, MOD)
for i in range(MAXN - 1, 0, -1):
    invfact[i - 1] = invfact[i] * i % MOD

def nCr(n, r):
    if r < 0 or r > n:
        return 0
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD

def nPr(n, r):
    if r < 0 or r > n:
        return 0
    return fact[n] * invfact[n - r] % MOD
```

$$C(n, r) = \frac{n!}{r!(n-r)!} \quad P(n, r) = \frac{n!}{(n-r)!}$$

Tip: The backward loop gets all inverse factorials from a single `pow`. If no modulus is required at all, `math.comb(n, r)` is exact and instant.

■ Combinatorial Identities

- $C(n, r) = C(n-1, r-1) + C(n-1, r)$ — Pascal.

- $\sum_{r=0}^n C(n, r) = 2^n$.
- Stars and bars: $x_1 + \dots + x_k = n, x_i \geq 0 \rightarrow C(n + k - 1, k - 1)$.
- Catalan: $C_n = \frac{1}{n+1} \binom{2n}{n}$ — balanced brackets, binary trees, triangulations.

■ Sieve of Eratosthenes $\mathcal{O}(n \log \log n)$

```
def sieve(n):
    is_prime = bytearray([1]) * (n + 1)
    is_prime[0:2] = b'\x00\x00'
    for i in range(2, isqrt(n) + 1):
        if is_prime[i]:
            # slice assignment: the loop runs in C
            is_prime[i*i::i] = bytearray(
                len(is_prime[i*i::i]))
    primes = [i for i in range(2, n + 1) if is_prime[i]]
    return is_prime, primes
```

Tip: The slice assignment is what makes this fast in Python. A plain inner `for` loop is several times slower, and `bytearray` uses 1 byte per entry instead of 8.

■ Linear Sieve with SPF $\mathcal{O}(n)$

```
def linear_sieve(n):
    spf = [0] * (n + 1)
    primes = []
    for i in range(2, n + 1):
        if spf[i] == 0:
            spf[i] = i
            primes.append(i)
        for p in primes:
            if p > spf[i] or i * p > n:
                break
            spf[i * p] = p
    return spf, primes

def factorize_spf(x, spf):
    #  $\mathcal{O}(\log x)$  after the sieve
    f = []
    while x > 1:
        f.append(spf[x])
        x //= spf[x]
    return f
```

■ Trial Division Factorization $\mathcal{O}(\sqrt{n})$

```
def factorize(x):
    f = []
    p = 2
    while p * p <= x:
        if x % p == 0:
            e = 0
            while x % p == 0:
                x //= p
                e += 1
            f.append((p, e))
        p += 1
    if x > 1:
        f.append((x, 1))
    return f
```

■ All Divisors $\mathcal{O}(\sqrt{n})$

```
def divisors(n):
    small, large = [], []
    for i in range(1, isqrt(n) + 1):
        if n % i == 0:
            small.append(i)
            if i != n // i:
                large.append(n // i)
    return small + large[::-1] # already sorted
```

From $n = \prod p_i^{e_i}$:

- Number of divisors: $\prod (e_i + 1)$.
- Sum of divisors: $\prod \frac{p_i^{e_i+1} - 1}{p_i - 1}$.

■ Divisor Sieve $\mathcal{O}(n \log n)$

```
divs = [[] for _ in range(n + 1)]
for i in range(1, n + 1):
    for j in range(i, n + 1, i):
        divs[j].append(i)
```

■ Euler's Totient Function

$\phi(n)$ counts integers in $[1, n]$ coprime to n .

```
def phi_single(n):
    #  $\mathcal{O}(\sqrt{n})$ 
    res, p = n, 2
    while p * p <= n:
        if n % p == 0:
            while n % p == 0:
                n //= p
            res -= res // p
        p += 1
    if n > 1:
        res -= res // n
    return res

def phi_sieve(n):
    #  $\mathcal{O}(n \log \log n)$ 
    phi = list(range(n + 1))
    for i in range(2, n + 1):
        if phi[i] == i:
            # i is prime
            for j in range(i, n + 1, i):
                phi[j] -= phi[j] // i
    return phi
```

Use: Euler's theorem: $a^{\phi(m)} \equiv 1 \pmod{m}$ when $\gcd(a, m) = 1$ — how you reduce enormous exponents modulo a composite.

■ Miller-Rabin Primality $\mathcal{O}(k \log^3 n)$

```
def is_prime(n):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:
```

```

        return n == p

d, s = n - 1, 0
while d % 2 == 0:
    d //= 2
    s += 1

for a in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
    x = pow(a, d, n)
    if x == 1 or x == n - 1:
        continue
    for _ in range(s - 1):
        x = x * x % n
        if x == n - 1:
            break
    else:
        return False
return True

```

Use: deterministic for every 64-bit n with this witness set.

Tip: The `for/else` is doing real work: `else` runs only when the loop finished without `break`, i.e. no witness cleared a .

■ Pollard's Rho Factorization $\mathcal{O}(n^{1/4})$

```

import random
from math import gcd

def pollard(n):
    if n % 2 == 0:
        return 2
    while True:
        x = random.randrange(2, n)
        y, c, d = x, random.randrange(1, n), 1
        while d == 1:
            x = (x * x + c) % n
            y = (y * y + c) % n
            y = (y * y + c) % n
            d = gcd(abs(x - y), n)
        if d != n:
            return d

def factor(n):
    if n == 1:
        return []
    if is_prime(n):
        return [n]
    d = pollard(n)
    return factor(d) + factor(n // d)

```

Use: factoring values up to 10^{18} where trial division would take hours.

■ Chinese Remainder Theorem $\mathcal{O}(\log m)$

```

def crt(a1, m1, a2, m2):
    g, p, _ = extgcd(m1, m2)
    if (a2 - a1) % g:
        return None  # incompatible
    lcm = m1 // g * m2
    t = (a2 - a1) // g * p % (m2 // g)
    return (a1 + t * m1) % lcm, lcm

```

■ Matrix Exponentiation $\mathcal{O}(k^3 \log n)$

```

def mat_mul(A, B, mod=MOD):
    n, m, k = len(A), len(B[0]), len(B)
    C = [[0] * m for _ in range(n)]
    for i in range(n):
        Ai, Ci = A[i], C[i]
        for p in range(k):
            if Ai[p]:
                v, Bp = Ai[p], B[p]
                for j in range(m):
                    Ci[j] = (Ci[j] + v * Bp[j]) % mod
    return C

def mat_pow(A, e, mod=MOD):
    n = len(A)
    R = [[int(i == j) for j in range(n)] for i in range(n)]
    while e:
        if e & 1:
            R = mat_mul(R, A, mod)
        A = mat_mul(A, A, mod)
        e >>= 1
    return R

```

Use: any linear recurrence at a huge index. Fibonacci: $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$ yields F_n in $\mathcal{O}(\log n)$.

■ Notable Sums

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2}\right)^2 \quad \sum_{i=0}^n r^i = \frac{r^{n+1} - 1}{r - 1}$$

```

s1 = n * (n + 1) // 2
s2 = n * (n + 1) * (2 * n + 1) // 6
s3 = (n * (n + 1) // 2) ** 2

```

Careful: Use `//`, not `/`. True division returns a float and loses precision past 2^{53} .

■ Prefix Sum 1D $\mathcal{O}(n \text{ build, } 1 \text{ query})$

```
from itertools import accumulate

pref = [0] + list(accumulate(a))    # fastest

def range_sum(l, r):                # inclusive, 0-indexed
    return pref[r + 1] - pref[l]
```

Tip: `accumulate` runs in C. The manual loop below is identical in meaning but noticeably slower:

```
pref = [0] * (n + 1)
for i in range(n):
    pref[i + 1] = pref[i] + a[i]
```

■ Prefix XOR

```
from operator import xor
px = [0] + list(accumulate(a, xor))

def range_xor(l, r):
    return px[r + 1] ^ px[l]
```

■ Prefix Sum 2D $\mathcal{O}(nm \text{ build, } 1 \text{ query})$

```
p = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(1, n + 1):
    row, prev, cur = a[i - 1], p[i - 1], p[i]
    for j in range(1, m + 1):
        cur[j] = row[j - 1] + prev[j] + cur[j - 1] - prev[j - 1]

def rect_sum(x1, y1, x2, y2):        # inclusive, 0-indexed
    x1 += 1; y1 += 1; x2 += 1; y2 += 1
    return (p[x2][y2] - p[x1 - 1][y2]
            - p[x2][y1 - 1] + p[x1 - 1][y1 - 1])
```

Tip: Binding `p[i-1]` and `p[i]` to locals before the inner loop removes two index lookups per iteration — a real speedup in CPython.

■ Difference Array 1D $\mathcal{O}(1 \text{ per update})$

```
d = [0] * (n + 1)

def add_range(l, r, v):              # inclusive
    d[l] += v
    if r + 1 <= n:
        d[r + 1] -= v

# Rebuild once, at the end
cur = 0
for i in range(n):
    cur += d[i]
    a[i] += cur
```

Use: many range updates, one read at the end. The exact mirror of a prefix sum.

■ Difference Array 2D

```
d = [[0] * (m + 2) for _ in range(n + 2)]

def add_rect(x1, y1, x2, y2, v):
    d[x1][y1] += v
    d[x1][y2 + 1] -= v
    d[x2 + 1][y1] -= v
    d[x2 + 1][y2 + 1] += v
# Then take a 2D prefix sum of d
```

■ Binary Search $\mathcal{O}(\log n)$

```
l, r, ans = 0, n - 1, -1
while l <= r:
    mid = (l + r) // 2
    if a[mid] == x:
        ans = mid
        break
    if a[mid] < x:
        l = mid + 1
    else:
        r = mid - 1
```

Use: Python integers never overflow, so `(l + r) // 2` is safe here — unlike in C++.

■ Binary Search on the Answer $\mathcal{O}(\log(\text{range}) \cdot f)$

```
lo, hi, ans = 0, 10**18, -1
while lo <= hi:
    mid = (lo + hi) // 2
    if check(mid):
        ans = mid
        hi = mid - 1
    else:
        lo = mid + 1
```

Use: "minimise the maximum", "maximise the minimum", and anything with a monotone predicate.

■ First True Pattern

```
lo, hi = 0, n                    # half-open [lo, hi)
while lo < hi:
    mid = (lo + hi) // 2
    if ok(mid):
        hi = mid
    else:
        lo = mid + 1
# lo == first index where ok() holds, or n
```

Tip: Far harder to get wrong than the `l <= r` form. Learn this one shape and reuse it.

■ Binary Search on Floats

```
lo, hi = 0.0, 1e9
```

```
for _ in range(100):          # fixed iteration count
    mid = (lo + hi) / 2
    if check(mid):
        hi = mid
    else:
        lo = mid
```

Tip: A fixed count cannot loop forever, and 100 halvings exceed any precision a problem will ask for.

■ bisect $\mathcal{O}(\log n)$

```
from bisect import bisect_left, bisect_right, insert

i = bisect_left(a, x)        # first index with a[i] >= x
j = bisect_right(a, x)       # first index with a[i] > x

count_x = bisect_right(a, x) - bisect_left(a, x)
exists = i < len(a) and a[i] == x

# Largest element <= x
k = bisect_right(a, x) - 1
if k >= 0:
    val = a[k]

insert(a, x)                 # insert, keeping sorted:  $\mathcal{O}(n)$ 
```

Careful: `insert` searches in $\mathcal{O}(\log n)$ but *shifts* in $\mathcal{O}(n)$. For many insertions use a heap, a BIT, or offline sorting.

Tip: `bisect` takes a `key=` argument in 3.10+. On older judges, search a separate list of keys.

■ Ternary Search $\mathcal{O}(\log(\text{range}))$

```
lo, hi = 0.0, 1e9
for _ in range(200):
    m1 = lo + (hi - lo) / 3
    m2 = hi - (hi - lo) / 3
    if f(m1) < f(m2):          # maximising
        lo = m1
    else:
        hi = m2
best = f(lo)
```

Use: strictly unimodal functions — one peak, no flat stretch.

■ Two Pointers $\mathcal{O}(n)$

```
l = 0
cur = 0
best = 0
for r in range(n):
    cur += a[r]                # extend
    while cur > K:              # shrink while invalid
        cur -= a[l]
        l += 1
    best = max(best, r - l + 1)
```

Careful: Requires monotonicity: once the window is invalid, extending right must not make it

valid again.

■ Sliding Window Maximum $\mathcal{O}(n)$

```
from collections import deque

def window_max(a, k):
    dq, res = deque(), []      # dq holds indices
    for i, x in enumerate(a):
        if dq and dq[0] <= i - k:
            dq.popleft()        # left the window
        while dq and a[dq[-1]] <= x:
            dq.pop()            # dominated
        dq.append(i)
        if i >= k - 1:
            res.append(a[dq[0]])
    return res
```

Tip: The deque stays decreasing, so the front is the window maximum. Flip the comparison for a minimum.

■ Kadane: Max Subarray Sum $\mathcal{O}(n)$

```
def kadane(a):
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

Careful: Seeding `best = 0` silently returns 0 on all-negative input. Seed from `a[0]`.

■ LIS $\mathcal{O}(n \log n)$

```
from bisect import bisect_left, bisect_right

def lis_length(a):
    t = []
    for x in a:
        i = bisect_left(t, x)
        if i == len(t):
            t.append(x)
        else:
            t[i] = x
    return len(t)
```

Careful: `t` is not the LIS itself — only its length is meaningful. Reconstruct with parent pointers:

```
def lis_reconstruct(a):
    t, pos, par = [], [], [-1] * len(a)
    for i, x in enumerate(a):
        j = bisect_left(t, x)
        if j > 0:
            par[i] = pos[j - 1]
        if j == len(t):
            t.append(x); pos.append(i)
        else:
            t[j] = x; pos[j] = i
    res, i = [], pos[-1]
```

```
while i != -1:
    res.append(a[i])
    i = par[i]
return res[::-1]
```

Tip: Use `bisect_right` for the longest *non-decreasing* subsequence.

Coordinate Compression $\mathcal{O}(n \log n)$

```
vals = sorted(set(a))
comp = {x: i for i, x in enumerate(vals)}
b = [comp[x] for x in a]
# vals[i] recovers the original value
```

Use: values up to 10^9 but only 10^5 distinct ones — compress to $[0, n)$ so they can index a BIT or segment tree.

Counting Inversions $\mathcal{O}(n \log n)$

```
def count_inversions(a):
    vals = sorted(set(a))
    comp = {x: i + 1 for i, x in enumerate(vals)}
```

5 Bit Manipulation

Basic Operations

```
mask | (1 << i)      # set bit i
mask & (1 << i)      # test bit i (nonzero if set)
mask ^ (1 << i)      # toggle bit i
mask & ~(1 << i)     # clear bit i

(mask >> i) & 1      # test, as a clean 0/1
```

Use: Python integers are arbitrary width, so $1 \ll 200$ is fine. None of the C++ 1LL caveats apply.

Least Significant Bit

```
mask & -mask        # isolate the lowest set bit
mask & (mask - 1)    # clear the lowest set bit
mask | (mask + 1)    # set the lowest clear bit
```

Python Bit Built-ins

```
x.bit_count()        # popcount, Python 3.10+
bin(x).count('1')     # popcount, any version
x.bit_length()        # bits needed for x
x.bit_length() - 1    # floor(log2(x)), for x > 0
(x & -x).bit_length() - 1 # index of the lowest set bit
```

```
bit = Fenwick(len(vals))
inv = 0
for x in reversed(a):
    c = comp[x]
    inv += bit.sum(c - 1) # already-seen smaller
    bit.add(c, 1)
return inv
```

Use: the minimum number of adjacent swaps that sorts the array.

Monotonic Stack $\mathcal{O}(n)$

```
# Next strictly greater element to the right
def next_greater(a):
    n = len(a)
    res, st = [-1] * n, [] # st holds indices
    for i, x in enumerate(a):
        while st and a[st[-1]] < x:
            res[st.pop()] = i
        st.append(i)
    return res
```

Use: previous/next greater or smaller, largest rectangle in a histogram, sum of subarray minimums.

```
int('1011', 2)      # parse binary
bin(11)              # '0b1011'
format(11, '08b')    # '00001011'
```

Careful: `x.bit_count()` needs 3.10. Older judges need `bin(x).count('1')`.

Common Tricks

```
(1 << n) - 1        # n low bits all set
n > 0 and n & (n - 1) == 0 # is a power of two
1 << (n - 1).bit_length() # next power of two
x & 1                # odd?
x >> 1               # floor division by 2
```

Use: unlike C++, $\gg$ on a negative Python int is a true *floor* shift: $-7 \gg 1 == -4$.

XOR Properties

- $a \oplus a = 0$ and $a \oplus 0 = a$.
- Commutative and associative — order never matters.
- If $a \oplus b = c$ then $a \oplus c = b$.
- XOR of $1..n$ has period 4: $n, 1, n+1, 0$ for $n \bmod 4 = 0, 1, 2, 3$.

Use: "every value appears twice except one" — XOR everything and the pairs cancel.

■ Iterate All Subsets $\mathcal{O}(2^n \cdot n)$

```
for mask in range(1 << n):
    for j in range(n):
        if mask & (1 << j):
            pass          # element j is in this subset

# Or with itertools, at C speed
from itertools import combinations
for k in range(n + 1):
    for sub in combinations(a, k):
        pass
```

■ Iterate Submasks $\mathcal{O}(3^n \text{ total})$

```
sub = mask
while sub:
    # sub is a nonempty submask of mask
    sub = (sub - 1) & mask

# Including the empty submask
sub = mask
while True:
    # process sub
    if sub == 0:
        break
    sub = (sub - 1) & mask
```

Tip: Summed over every mask this runs 3^n times, not 4^n — which is what makes subset DP feasible at $n \leq 20$.

■ Subsets of Size K (Gosper's Hack) $\mathcal{O}\left(\binom{n}{k}\right)$

```
cur = (1 << k) - 1
while cur < (1 << n):
    # process cur: exactly k bits set
    c = cur & -cur
    r = cur + c
    cur = ((r ^ cur) >> 2) // c | r
```

Careful: Use `//`, not `/`. True division returns a float and breaks the bit arithmetic.

■ SOS DP (Sum over Subsets) $\mathcal{O}(2^n \cdot n)$

```
f = a[:]          # f[mask] = a[mask]
for i in range(n):
    bit = 1 << i
    for mask in range(1 << n):
        if mask & bit:
            f[mask] += f[mask ^ bit]
# f[mask] = sum of a[sub] over every submask sub of mask
```

Use: aggregating over all submasks in $\mathcal{O}(2^n n)$ instead of $\mathcal{O}(3^n)$. Reverse the condition to sum over supersets.

■ XOR Linear Basis $\mathcal{O}(n \log C)$

```
class Basis:
    B = 60

    def __init__(self):
        self.b = [0] * (self.B + 1)

    def insert(self, x):
        for i in range(self.B, -1, -1):
            if not (x >> i) & 1:
                continue
            if not self.b[i]:
                self.b[i] = x
                return True
            x ^= self.b[i]
        return False          # already representable

    def max_xor(self):
        res = 0
        for i in range(self.B, -1, -1):
            res = max(res, res ^ self.b[i])
        return res
```

Use: maximum XOR of a subset, counting distinct achievable XORs (2^{rank}), and testing reachability of a value.

6 Strings

■ Reading and Splitting

```
s = input().rstrip()          # drop the trailing newline
a, b, c = map(int, input().split())
nums = list(map(int, input().split()))

grid = [input().rstrip() for _ in range(n)]
```

Careful: With `input = sys.stdin.readline`, the newline survives. Forgetting `.rstrip()` makes string comparisons fail in ways that are painful to spot.

■ String Utilities

```
s[l:r]          # chars l .. r-1 (r excluded)
s[::-1]         # reversed
s.find("abc")    # -1 when absent
s.count("ab")    # non-overlapping occurrences
s.replace("a", "b")
s.startswith("ab") ; s.endswith("z")
"-".join(words)
"".join(sorted(s)) # anagram normal form

ord('a')        # 97
```

```
chr(97)          # 'a'
ord(c) - ord('a') # 0..25 index
s.isalpha() ; s.isdigit() ; s.islower()
s.upper() ; s.lower()
```

Careful: Strings are immutable. Building one with `s += ch` in a loop is $O(n^2)$ — append to a list and `"".join()` once.

■ Character Frequency

```
from collections import Counter

freq = Counter(s)
is_anagram = Counter(a) == Counter(b)

cnt = [0] * 26          # faster than Counter
for ch in s:
    cnt[ord(ch) - 97] += 1
```

■ Z-Function $O(n)$

`z[i]` = length of the longest common prefix of `s` and `s[i:]`.

```
def z_function(s):
    n = len(s)
    z = [0] * n
    l = r = 0
    for i in range(1, n):
        if i <= r:
            z[i] = min(r - i + 1, z[i - l])
            while i + z[i] < n and s[z[i]] == s[i + z[i]]:
                z[i] += 1
            if i + z[i] - 1 > r:
                l, r = i, i + z[i] - 1
    return z

def z_search(text, pat):
    s = pat + '#' + text          # '#' occurs in neither
    z = z_function(s)
    m = len(pat)
    return [i - m - 1
            for i in range(m + 1, len(s)) if z[i] == m]
```

Careful: The separator must not occur in either string, or matches can straddle the join.

■ KMP Prefix Function $O(n)$

`pi[i]` = longest proper prefix of `s[:i+1]` that is also a suffix of it.

```
def prefix_function(s):
    n = len(s)
    pi = [0] * n
    for i in range(1, n):
        j = pi[i - 1]
        while j > 0 and s[i] != s[j]:
            j = pi[j - 1]
        if s[i] == s[j]:
            j += 1
        pi[i] = j
    return pi
```

```
def kmp_search(text, pat):
    s = pat + '#' + text
    pi = prefix_function(s)
    m = len(pat)
    return [i - 2 * m
            for i in range(m + 1, len(s)) if pi[i] == m]
```

Use: `n - pi[n-1]` is the smallest period of the string; it repeats exactly when that period divides `n`.

Tip: For plain substring search with no extra structure needed, `text.find(pat)` is implemented in C and beats a Python KMP.

■ Manacher $O(n)$

```
def manacher_odd(s):          # d1[i] = odd radius at i
    n = len(s)
    d1 = [0] * n
    l, r = 0, -1
    for i in range(n):
        k = 1 if i > r else min(d1[l + r - i], r - i + 1)
        while 0 <= i - k and i + k < n and s[i-k] == s[i+k]:
            k += 1
        d1[i] = k
        if i + k - 1 > r:
            l, r = i - k + 1, i + k - 1
    return d1

def manacher_even(s):         # d2[i] = even radius at i-1/i
    n = len(s)
    d2 = [0] * n
    l, r = 0, -1
    for i in range(n):
        k = 0 if i > r else min(d2[l + r - i + 1], r - i + 1)
        while (0 <= i - k - 1 and i + k < n
               and s[i - k - 1] == s[i + k]):
            k += 1
        d2[i] = k
        if i + k - 1 > r:
            l, r = i - k, i + k - 1
    return d2

def longest_palindrome(s):
    d1, d2 = manacher_odd(s), manacher_even(s)
    return max(max((2 * x - 1) for x in d1),
               max((2 * x) for x in d2), default=0)
```

Use: the total number of palindromic substrings is $\sum d1[i] + \sum d2[i]$.

■ String Hashing $O(n)$ build, 1 per substring

```
import random

class Hash:
    def __init__(self, s, mod=(1 << 61) - 1):
        self.mod = mod
        self.B = random.randrange(256, mod - 1)
        n = len(s)
        self.h = [0] * (n + 1)
        self.p = [1] * (n + 1)
```

```

for i, ch in enumerate(s):
    self.h[i+1] = (self.h[i] * self.B + ord(ch)) % mod
    self.p[i+1] = self.p[i] * self.B % mod

def get(self, l, r):
    # inclusive [l, r]
    return (self.h[r+1]
            - self.h[l] * self.p[r-l+1]) % self.mod

```

Use: $O(1)$ substring comparison, so the longest common prefix of two suffixes takes $O(\log n)$ by binary search.

Careful: Randomise the base — a fixed base and a small modulus is trivially hacked. Python's big integers make $2^{61} - 1$ effortless here; no `__int128` trickery needed.

■ Trie $O(|s| \text{ per operation})$

```

class Trie:
    def __init__(self):
        self.nxt = [{}]
```

node -> {char: node}

```

        self.cnt = [0]
        # words ending at node

    def _new(self):
        self.nxt.append({})

```

```

        self.cnt.append(0)
        return len(self.nxt) - 1

def insert(self, s):
    v = 0
    for ch in s:
        if ch not in self.nxt[v]:
            self.nxt[v][ch] = self._new()
            v = self.nxt[v][ch]
        self.cnt[v] += 1

def contains(self, s):
    v = 0
    for ch in s:
        if ch not in self.nxt[v]:
            return False
        v = self.nxt[v][ch]
    return self.cnt[v] > 0

```

Use: prefix queries, autocomplete, and — over the bits of each number — maximum XOR pair queries.

Tip: Flat lists indexed by node id beat a class-per-node design in Python: fewer objects, far less allocation.

7 Graphs: Traversal and Ordering

■ Representations

```

# Adjacency list, 0-indexed
adj = [[] for _ in range(n)]
adj[u].append(v)
adj[v].append(u)
# omit if directed

# Weighted
wadj = [[] for _ in range(n)]
wadj[u].append((v, w))

# Edge list: Kruskal, Bellman-Ford
edges = []
# (w, u, v)

# Sparse / non-integer labels
from collections import defaultdict
adj = defaultdict(list)

```

Careful: `[[]] * n` makes n references to one list. Use the comprehension.

■ Graph Identities

- Tree on n nodes: exactly $n - 1$ edges, connected, acyclic.
- Complete graph K_n : $\frac{n(n-1)}{2}$ edges.
- Handshake lemma: $\sum_v \deg(v) = 2|E|$.
- Euler circuit iff every degree is even; Euler path iff exactly two degrees are odd.
- Bipartite iff no odd-length cycle.

■ DFS (Iterative) $O(V + E)$

```

def dfs(s, adj, n):
    vis = [False] * n
    st = [s]
    vis[s] = True
    while st:
        u = st.pop()
        for v in adj[u]:
            if not vis[v]:
                vis[v] = True
                st.append(v)
    return vis

```

Tip: Mark on *push*, not on *pop*, or a node can be stacked many times.

Careful: Prefer this over recursion in Python. Recursive DFS on 10^5 nodes will hit the recursion limit or blow the C stack.

■ DFS with Entry and Exit

```

# When you need post-order work (subtree sizes, etc.)
def dfs_order(root, adj, n):
    par = [-1] * n
    order = []
    st = [root]
    seen = [False] * n
    seen[root] = True
    while st:
        u = st.pop()

```

```

order.append(u)
for v in adj[u]:
    if not seen[v]:
        seen[v] = True
        par[v] = u
        st.append(v)
return order, par
# Iterate reversed(order) for post-order processing

```

Tip: This is the standard workaround for tree DP without recursion: walk `reversed(order)` and every child is already finished.

■ BFS and Distances $\mathcal{O}(V + E)$

```

from collections import deque

def bfs(s, adj, n):
    dist = [-1] * n
    q = deque([s])
    dist[s] = 0
    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

```

Use: shortest paths on *unweighted* graphs. `dist[v] == -1` doubles as the visited flag.

■ Multi-Source BFS

```

q = deque()
for s in sources:
    dist[s] = 0
    q.append(s)
# ... the same BFS loop from here

```

Use: "distance to the nearest fire / gate / exit". One pass beats one BFS per source.

■ 0-1 BFS $\mathcal{O}(V + E)$

```

from collections import deque

def bfs01(s, wadj, n):
    dist = [float('inf')] * n
    dist[s] = 0
    dq = deque([s])
    while dq:
        u = dq.popleft()
        for v, w in wadj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return dist

```

Use: Dijkstra without the log when every weight is 0 or 1.

■ Grid Traversal

```

DIRS4 = ((-1, 0), (1, 0), (0, -1), (0, 1))
DIRS8 = ((-1,-1), (-1,0), (-1,1), (0,-1),
          (0,1), (1,-1), (1,0), (1,1))

for dx, dy in DIRS4:
    nx, ny = x + dx, y + dy
    if not (0 <= nx < R and 0 <= ny < C):
        continue
    if grid[nx][ny] == '#':
        continue
# ...

```

Tip: Flatten a cell to $x * C + y$ when you need a single integer id (for a DSU or a flat `dist` list). It is faster than tuples.

■ Connected Components $\mathcal{O}(V + E)$

```

comp = [-1] * n
c = 0
for i in range(n):
    if comp[i] == -1:
        # BFS or iterative DFS from i, labelling comp[] = c
        c += 1

```

■ Bipartite Check $\mathcal{O}(V + E)$

```

from collections import deque

def bipartite(adj, n):
    color = [-1] * n
    for s in range(n):
        if color[s] != -1:
            continue
        color[s] = 0
        q = deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if color[v] == -1:
                    color[v] = color[u] ^ 1
                    q.append(v)
                elif color[v] == color[u]:
                    return False, color
    return True, color

```

Careful: Loop over every start node — checking only node 0 misses disconnected components.

■ Greedy Coloring $\mathcal{O}(V + E)$

```

def greedy_coloring(adj, n):
    color = [-1] * n
    for u in range(n):
        used = {color[v] for v in adj[u] if color[v] != -1}

```

```

c = 0
while c in used:
    c += 1
color[u] = c
return color

```

Careful: Valid, but not minimal. Minimum graph coloring is NP-hard.

■ Cycle in an Undirected Graph $\mathcal{O}(V + E)$

```

def has_cycle_undirected(adj, n):
    vis = [False] * n
    for s in range(n):
        if vis[s]:
            continue
        st = [(s, -1)]
        vis[s] = True
        while st:
            u, p = st.pop()
            for v in adj[u]:
                if not vis[v]:
                    vis[v] = True
                    st.append((v, u))
                elif v != p:
                    return True
            # note: with multi-edges, track edge ids instead
    return False

```

■ Cycle in a Directed Graph $\mathcal{O}(V + E)$

```

# 0 = unvisited, 1 = on the current path, 2 = done
def has_cycle_directed(adj, n):
    state = [0] * n
    for s in range(n):
        if state[s]:
            continue
        st = [(s, iter(adj[s]))]
        state[s] = 1
        while st:
            u, it = st[-1]

```

```

        found = False
        for v in it:
            if state[v] == 1:
                return True
            if state[v] == 0:
                state[v] = 1
                st.append((v, iter(adj[v])))
                found = True
                break
        if not found:
            state[u] = 2
            st.pop()
    return False

```

Careful: State 2 is what separates a back edge (a real cycle) from a cross edge. A plain visited array reports cycles that do not exist.

■ Topological Sort (Kahn) $\mathcal{O}(V + E)$

```

from collections import deque

def topo_sort(adj, n):
    indeg = [0] * n
    for u in range(n):
        for v in adj[u]:
            indeg[v] += 1

    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)

    return order if len(order) == n else [] # [] = cycle

```

Use: a short result means a cycle exists, so this doubles as cycle detection. Use `heapq` instead of `deque` for the lexicographically smallest order.

8 Shortest Paths, MST and DSU

■ Dijkstra $\mathcal{O}(E \log V)$

```

from heapq import heappush, heappop

def dijkstra(wadj, s, n):
    INF = float('inf')
    dist = [INF] * n
    dist[s] = 0
    pq = [(0, s)]
    while pq:
        d, u = heappop(pq)
        if d > dist[u]:
            continue # stale entry
        for v, w in wadj[u]:

```

```

            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heappush(pq, (nd, v))
    return dist

```

Careful: Fails on negative weights. The `d > dist[u]` skip is not optional — Python's `heapq` has no decrease-key, so stale entries must be discarded on pop.

■ Path Reconstruction

```

par = [-1] * n

```

```
# inside the relaxation:
#   par[v] = u
```

```
path, cur = [], t
while cur != -1:
    path.append(cur)
    cur = par[cur]
path.reverse()
```

■ Bellman-Ford $\mathcal{O}(VE)$

```
def bellman_ford(edges, s, n):
    INF = float('inf')
    dist = [INF] * n
    dist[s] = 0
    for _ in range(n - 1):
        changed = False
        for w, u, v in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break # early exit
    # one more pass: any improvement means a negative cycle
    neg = any(dist[u] + w < dist[v] for w, u, v in edges)
    return dist, neg
```

Use: negative weights, and detecting negative cycles. The `changed` flag often cuts the real runtime dramatically.

■ Floyd-Warshall $\mathcal{O}(V^3)$

```
# d[i][j] = edge weight, INF if none, 0 on the diagonal
for k in range(n):
    dk = d[k]
    for i in range(n):
        dik = d[i][k]
        if dik == INF:
            continue # big speedup
        di = d[i]
        for j in range(n):
            nd = dik + dk[j]
            if nd < di[j]:
                di[j] = nd

# d[i][i] < 0 => i lies on a negative cycle
```

Careful: `k` must be the outermost loop. Any other order silently gives wrong answers.

Tip: In Python, prefer the row-wise form: `d[i] = list(map(min, d[i], (dik + x for x in d[k])))` pushes the inner loop into C and is several times faster.

■ DSU / Union-Find $\mathcal{O}(\alpha(n))$

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.sz = [1] * n
        self.components = n
```

```
def find(self, x):
    # iterative + halving
    while self.parent[x] != x:
        self.parent[x] = self.parent[self.parent[x]]
        x = self.parent[x]
    return x

def unite(self, a, b):
    # union by size
    ra, rb = self.find(a), self.find(b)
    if ra == rb:
        return False
    if self.sz[ra] < self.sz[rb]:
        ra, rb = rb, ra
    self.parent[rb] = ra
    self.sz[ra] += self.sz[rb]
    self.components -= 1
    return True

def same(self, a, b):
    return self.find(a) == self.find(b)

def size(self, x):
    return self.sz[self.find(x)]
```

Tip: The iterative `find` with path halving avoids Python's recursion overhead entirely and is the right default here.

Careful: DSU cannot remove edges. When a problem deletes them, process the queries in reverse.

■ Kruskal MST $\mathcal{O}(E \log E)$

```
def kruskal(edges, n):
    edges.sort() # by weight
    dsu = DSU(n)
    total = used = 0
    for w, u, v in edges:
        if dsu.unite(u, v):
            total += w
            used += 1
            if used == n - 1:
                break
    return total if used == n - 1 else -1 # -1: disconnected
```

Tip: Sort descending for a maximum spanning tree. For "minimise the largest edge on a path" problems, the MST is the answer.

■ Prim MST $\mathcal{O}(E \log V)$

```
from heapq import heappush, heappop

def prim(wadj, n):
    inmst = [False] * n
    pq = [(0, 0)]
    total = used = 0
    while pq and used < n:
        w, u = heappop(pq)
        if inmst[u]:
            continue
        inmst[u] = True
        total += w
        used += 1
```

```
for v, c in wadj[u]:
    if not inmst[v]:
        heappush(pq, (c, v))
```

```
return total if used == n else -1
```

Use: better than Kruskal on dense graphs, where sorting all E edges dominates.

9 Trees and Advanced Graphs

■ Rooting: Parent, Depth, Subtree Size $\mathcal{O}(n)$

```
def root_tree(adj, n, root=0):
    par = [-1] * n
    dep = [0] * n
    order = [root]
    seen = [False] * n
    seen[root] = True
    for u in order:
        for v in adj[u]:
            if not seen[v]:
                seen[v] = True
                par[v] = u
                dep[v] = dep[u] + 1
                order.append(v)

    sub = [1] * n
    for u in reversed(order):
        if par[u] != -1:
            sub[par[u]] += sub[u]
    return par, dep, sub, order
```

Tip: Iterating a list while appending to it gives a clean, recursion-free BFS. `reversed(order)` is a valid post-order for any tree DP.

■ Tree Diameter $\mathcal{O}(n)$

```
# 1. BFS from any node -> farthest node a
# 2. BFS from a -> farthest node b
# dist(a, b) is the diameter
def diameter(adj, n):
    d = bfs(0, adj, n)
    a = max(range(n), key=lambda i: d[i])
    d = bfs(a, adj, n)
    b = max(range(n), key=lambda i: d[i])
    return d[b], a, b
```

Careful: The two-BFS trick is valid on *trees* only. It is wrong on general graphs.

■ LCA with Binary Lifting $\mathcal{O}(n \log n$ build, $\log n$ query)

```
class LCA:
    def __init__(self, adj, n, root=0):
        self.LOG = max(1, n.bit_length())
        self.par, self.dep, _, _ = root_tree(adj, n, root)
        self.par[root] = root  # self-loop at the root

        self.up = [self.par[:]]
        for j in range(1, self.LOG):
            prev = self.up[j-1]
```

```
        self.up.append([prev[prev[i]] for i in range(n)])

    def query(self, u, v):
        if self.dep[u] < self.dep[v]:
            u, v = v, u
        diff = self.dep[u] - self.dep[v]
        for j in range(self.LOG):
            if diff >> j & 1:
                u = self.up[j][u]
        if u == v:
            return u
        for j in range(self.LOG - 1, -1, -1):
            if self.up[j][u] != self.up[j][v]:
                u, v = self.up[j][u], self.up[j][v]
        return self.up[0][u]

    def dist(self, u, v):
        return (self.dep[u] + self.dep[v]
                - 2 * self.dep[self.query(u, v)])

    def kth_ancestor(self, u, k):
        for j in range(self.LOG):
            if k >> j & 1:
                u = self.up[j][u]
        return u
```

Careful: The root must point to itself, so lifting past it is a harmless no-op.

■ Euler Tour: Subtree as a Range $\mathcal{O}(n)$

```
def euler_tour(adj, n, root=0):
    tin = [0] * n
    tout = [0] * n
    timer = 0
    st = [(root, -1, False)]
    while st:
        u, p, back = st.pop()
        if back:
            tout[u] = timer  # half-open [tin, tout)
            continue
        tin[u] = timer
        timer += 1
        st.append((u, p, True))
        for v in adj[u]:
            if v != p:
                st.append((v, u, False))
    return tin, tout

def is_ancestor(u, v, tin, tout):
    return tin[u] <= tin[v] and tout[v] <= tout[u]
```

Use: a subtree becomes the contiguous range $[tin_u, tout_u)$, so subtree sum and subtree update reduce to an ordinary BIT or segment tree range operation.

■ DP on Trees $\mathcal{O}(n)$

```
# Largest independent set, iteratively
def max_independent_set(adj, n, root=0):
    par, dep, sub, order = root_tree(adj, n, root)
    take = [1] * n      # u taken
    skip = [0] * n      # u not taken
    for u in reversed(order):
        p = par[u]
        if p != -1:
            skip[p] += max(take[u], skip[u])
            take[p] += skip[u]
    return max(take[root], skip[root])
```

Tip: Pushing values into the parent while walking `reversed(order)` replaces the whole recursive formulation.

■ Bridges $\mathcal{O}(V + E)$

An edge whose removal disconnects the graph.

```
def bridges(adj_id, n):
    # adj_id[u] = list of (neighbour, edge_id)
    tin = [-1] * n
    low = [0] * n
    timer = 0
    res = []
    for s in range(n):
        if tin[s] != -1:
            continue
        st = [(s, -1, iter(adj_id[s]))]
        tin[s] = low[s] = timer
        timer += 1
        while st:
            u, pe, it = st[-1]
            advanced = False
            for v, eid in it:
                if eid == pe:
                    continue
                if tin[v] != -1:
                    low[u] = min(low[u], tin[v])
                else:
                    tin[v] = low[v] = timer
                    timer += 1
                    st.append((v, eid, iter(adj_id[v])))
                    advanced = True
                    break
            if not advanced:
                st.pop()
                if st:
                    p = st[-1][0]
                    low[p] = min(low[p], low[u])
```

```
        if low[u] > tin[p]:
            res.append((p, u))
    return res
```

Careful: Skip by *edge id*, never by parent node — otherwise parallel edges are wrongly reported as bridges.

■ SCC (Kosaraju) $\mathcal{O}(V + E)$

```
def kosaraju(adj, n):
    radj = [[] for _ in range(n)]
    for u in range(n):
        for v in adj[u]:
            radj[v].append(u)

    vis = [False] * n
    order = []
    for s in range(n):
        if vis[s]:
            continue
        st = [(s, iter(adj[s]))]
        vis[s] = True
        while st:
            u, it = st[-1]
            for v in it:
                if not vis[v]:
                    vis[v] = True
                    st.append((v, iter(adj[v])))
                    break
            else:
                order.append(u)
                st.pop()

    comp = [-1] * n
    c = 0
    for u in reversed(order):
        if comp[u] != -1:
            continue
        stack = [u]
        comp[u] = c
        while stack:
            x = stack.pop()
            for y in radj[x]:
                if comp[y] == -1:
                    comp[y] = c
                    stack.append(y)
        c += 1
    return comp, c
```

Use: contracting each SCC gives a DAG (the condensation) that you can topologically sort or run a DP over.

■ How to Build a DP

1. **State** — what exactly does `dp[i][j]` mean? Write it as a sentence first.
2. **Transition** — how do smaller states combine?
3. **Base case** — the smallest state, filled directly.
4. **Order** — iterate so every dependency is ready.
5. **Answer** — which state (or aggregate) is the result?

Tip: Cost = number of states $\times$ cost per transition. Check that against the constraints before writing code.

■ Top-Down vs Bottom-Up

```
from functools import cache

# Top-down: fastest to write, only visits reachable states
@cache
def f(i, cap):
    if i == n or cap == 0:
        return 0
    best = f(i + 1, cap)
    if wt[i] <= cap:
        best = max(best, f(i + 1, cap - wt[i]) + val[i])
    return best
```

Careful: On deep DPs the recursion limit bites. Convert to bottom-up, or run inside a thread with a big stack.

■ 0/1 Knapsack $\mathcal{O}(nW)$

```
dp = [0] * (W + 1)
for i in range(n):
    w, v = wt[i], val[i]
    for c in range(W, w - 1, -1):  # DESCENDING
        cand = dp[c - w] + v
        if cand > dp[c]:
            dp[c] = cand
# dp[W] = best value within capacity W
```

Careful: Descending is what enforces "each item at most once". Ascending turns it into the unbounded version.

■ Unbounded Knapsack / Coin Change $\mathcal{O}(nW)$

```
# Minimum number of coins summing to W
BIG = float('inf')
dp = [BIG] * (W + 1)
dp[0] = 0
for c in coins:
    for w in range(c, W + 1):  # ASCENDING
        if dp[w - c] + 1 < dp[w]:
            dp[w] = dp[w - c] + 1

# Number of distinct combinations (order ignored)
ways = [0] * (W + 1)
```

```
ways[0] = 1
for c in coins:
    for w in range(c, W + 1):
        ways[w] += ways[w - c]
```

Careful: Loop order decides the meaning. Coins outside, sums inside counts *combinations*; sums outside, coins inside counts *permutations*.

■ LCS $\mathcal{O}(nm)$

```
prev = [0] * (m + 1)
for i in range(1, n + 1):
    cur = [0] * (m + 1)
    ai = a[i - 1]
    for j in range(1, m + 1):
        if ai == b[j - 1]:
            cur[j] = prev[j - 1] + 1
        else:
            cur[j] = max(prev[j], cur[j - 1])
    prev = cur
# prev[m] = LCS length
```

■ Edit Distance $\mathcal{O}(nm)$

```
prev = list(range(m + 1))  # insertions
for i in range(1, n + 1):
    cur = [i] + [0] * m  # deletions
    ai = a[i - 1]
    for j in range(1, m + 1):
        if ai == b[j - 1]:
            cur[j] = prev[j - 1]
        else:
            cur[j] = 1 + min(prev[j],  # delete
                             cur[j - 1],  # insert
                             prev[j - 1])  # replace
    prev = cur
# prev[m] = edit distance
```

■ Bitmask DP (TSP) $\mathcal{O}(2^n n^2)$

```
INF = float('inf')
full = 1 << n
dp = [[INF] * n for _ in range(full)]
dp[1][0] = 0  # start at node 0

for mask in range(full):
    row = dp[mask]
    for i in range(n):
        if row[i] == INF or not mask >> i & 1:
            continue
        base, ci = row[i], cost[i]
        for j in range(n):
            if mask >> j & 1:
                continue
            nm = mask | (1 << j)
            if base + ci[j] < dp[nm][j]:
                dp[nm][j] = base + ci[j]

ans = min(dp[full - 1][i] + cost[i][0] for i in range(1, n))
```

Use: $n \leq 20$ in the constraints is the tell for bitmask DP.

Digit DP $\mathcal{O}(|d| \cdot 10 \cdot \text{states})$

```
from functools import lru_cache

def count_up_to(num_str):
    digits = list(map(int, num_str))

    @lru_cache(maxsize=None)
    def go(pos, tight, acc):
        if pos == len(digits):
            return 1 if good(acc) else 0
        lim = digits[pos] if tight else 9
        total = 0
        for d in range(lim + 1):
            total += go(pos + 1,
                        tight and d == lim,
                        acc + d)

        return total

    res = go(0, True, 0)
    go.cache_clear()
    return res

# Range [L, R]: count_up_to(R) - count_up_to(L - 1)
```

Tip: `lru_cache` handles the `tight` flag correctly on its own — it is part of the key. Just

remember to clear between tests.

Longest Path on a DAG $\mathcal{O}(V + E)$

```
dp = [0] * n
for u in topo_order:
    for v in adj[u]:
        if dp[u] + 1 > dp[v]:
            dp[v] = dp[u] + 1
```

Use: NP-hard on general graphs, linear on a DAG — the topological order removes all cyclic dependencies.

Memory Optimisation

```
# When row i only reads row i-1, keep two rows
prev = [0] * (m + 1)
for i in range(1, n + 1):
    cur = [0] * (m + 1)
    # fill cur from prev
    prev = cur
```

Tip: A $10^5 \times 10^5$ table is impossible; two rows is nothing. Rolling arrays are often the difference between MLE and AC.

11 Advanced Data Structures

Fenwick Tree (BIT) $\mathcal{O}(\log n)$

```
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)

    def add(self, i, v):          # 1-indexed
        while i <= self.n:
            self.bit[i] += v
            i += i & -i

    def sum(self, i):            # prefix [1, i]
        r = 0
        while i > 0:
            r += self.bit[i]
            i -= i & -i
        return r

    def range(self, l, r):
        return self.sum(r) - self.sum(l - 1)

    def kth(self, k):
        # smallest i with prefix sum >= k,  $\mathcal{O}(\log n)$ 
        pos = 0
        step = 1 << (self.n.bit_length() - 1)
        while step:
            nxt = pos + step
            if nxt <= self.n and self.bit[nxt] < k:
```

```
        pos = nxt
        k -= self.bit[pos]
        step >>= 1
    return pos + 1
```

Careful: A BIT is 1-indexed. Passing index 0 makes `add` loop forever because $0 \& -0 == 0$.

Tip: Smaller and faster than a segment tree. Reach for it whenever prefix sums suffice.

Fenwick: Range Update, Point Query $\mathcal{O}(\log n)$

```
# Store the difference array inside the BIT
def add_range(f, l, r, v):
    f.add(l, v)
    f.add(r + 1, -v)

def point_query(f, i):
    return f.sum(i)
```

2D Fenwick $\mathcal{O}(\log^2 n)$

```
class Fenwick2D:
    def __init__(self, n, m):
        self.n, self.m = n, m
        self.bit = [[0] * (m + 1) for _ in range(n + 1)]
```

```

def add(self, x, y, v):
    i = x
    while i <= self.n:
        j = y
        row = self.bit[i]
        while j <= self.m:
            row[j] += v
            j += j & -j
        i += i & -i

def sum(self, x, y):
    r = 0
    i = x
    while i > 0:
        j = y
        row = self.bit[i]
        while j > 0:
            r += row[j]
            j -= j & -j
        i -= i & -i
    return r

def rect(self, x1, y1, x2, y2):
    return (self.sum(x2, y2) - self.sum(x1 - 1, y2)
            - self.sum(x2, y1 - 1)
            + self.sum(x1 - 1, y1 - 1))

```

■ Segment Tree (Iterative) $\mathcal{O}(\log n)$

```

class SegTree:
    def __init__(self, a):
        self.n = len(a)
        self.t = [0] * (2 * self.n)
        self.t[self.n:] = a
        for i in range(self.n - 1, 0, -1):
            self.t[i] = self.t[2*i] + self.t[2*i + 1]

    def update(self, pos, val):    # point assignment
        pos += self.n
        self.t[pos] = val
        pos >>= 1
        while pos:
            self.t[pos] = self.t[2*pos] + self.t[2*pos + 1]
            pos >>= 1

    def query(self, l, r):        # inclusive [l, r]
        res = 0                  # neutral element
        l += self.n
        r += self.n + 1
        while l < r:
            if l & 1:
                res += self.t[l]
                l += 1
            if r & 1:
                r -= 1
                res += self.t[r]
            l >>= 1
            r >>= 1
        return res

```

Tip: Compact and the fastest segment tree in Python. For min/max, swap + for min/max and change the neutral element.

■ Segment Tree with Lazy Propagation $\mathcal{O}(\log n)$

```

class LazySeg:
    def __init__(self, a):
        self.n = len(a)
        self.t = [0] * (4 * self.n)
        self.lz = [0] * (4 * self.n)
        self._build(a, 1, 0, self.n - 1)

    def _build(self, a, v, tl, tr):
        if tl == tr:
            self.t[v] = a[tl]
            return
        tm = (tl + tr) // 2
        self._build(a, 2*v, tl, tm)
        self._build(a, 2*v + 1, tm + 1, tr)
        self.t[v] = self.t[2*v] + self.t[2*v + 1]

    def _push(self, v, tl, tr):
        if not self.lz[v]:
            return
        self.t[v] += self.lz[v] * (tr - tl + 1)
        if tl != tr:
            self.lz[2*v] += self.lz[v]
            self.lz[2*v + 1] += self.lz[v]
        self.lz[v] = 0

    def update(self, v, tl, tr, l, r, val):
        self._push(v, tl, tr)
        if l > tr or tl > r:
            return
        if l <= tl and tr <= r:
            self.lz[v] += val
            self._push(v, tl, tr)
            return
        tm = (tl + tr) // 2
        self.update(2*v, tl, tm, l, r, val)
        self.update(2*v + 1, tm + 1, tr, l, r, val)
        self.t[v] = self.t[2*v] + self.t[2*v + 1]

    def query(self, v, tl, tr, l, r):
        self._push(v, tl, tr)
        if l > tr or tl > r:
            return 0
        if l <= tl and tr <= r:
            return self.t[v]
        tm = (tl + tr) // 2
        return (self.query(2*v, tl, tm, l, r)
                + self.query(2*v + 1, tm + 1, tr, l, r))

```

Careful: `_push` must run at the top of *every* recursive call, queries included. A missing push is the classic lazy bug.

■ Sparse Table (RMQ) $\mathcal{O}(n \log n \text{ build, } 1 \text{ query})$

```

class SparseTable:
    def __init__(self, a):
        n = len(a)
        self.lg = [0] * (n + 1)
        for i in range(2, n + 1):
            self.lg[i] = self.lg[i >> 1] + 1

        self.sp = [a[:]]

```

```

j = 1
while (1 << j) <= n:
    prev = self.sp[-1]
    half = 1 << (j - 1)
    self.sp.append([
        min(prev[i], prev[i + half])
        for i in range(n - (1 << j) + 1)
    ])
    j += 1

def query(self, l, r):          # inclusive
    j = self.lg[r - l + 1]
    return min(self.sp[j][l], self.sp[j][r - (1 << j) + 1])

```

Careful: Only valid for *idempotent* operations — min, max, gcd — where overlap is harmless. Not for sums, and it does not support updates.

Ordered Set Substitute

Python has no built-in balanced BST. The usual replacements:

```

# 1. A BIT over compressed values (order statistics)
#    -> kth() and rank queries in O(log n)

# 2. sortedcontainers, when the judge provides it
from sortedcontainers import SortedList
sl = SortedList()
sl.add(x)
sl.remove(x)
sl[k]          # k-th smallest, O(log n)
sl.bisect_left(x) # rank of x

```

Careful: `sortedcontainers` is not in the standard library. Codeforces ships it; many judges do not. The BIT approach always works.

Coordinate Compression

```

vals = sorted(set(a))
comp = {x: i + 1 for i, x in enumerate(vals)} # 1-indexed

```

Use: the standard prerequisite for putting large values into a BIT.

12 Contest Techniques and Pitfalls

Meet in the Middle $\mathcal{O}(2^{n/2} \cdot n)$

```

from itertools import combinations
from bisect import bisect_right

def subset_sums(v):
    sums = [0]
    for x in v:
        sums += [s + x for s in sums]
    return sums

# n <= 40: split, sort one half, binary search it
left = sorted(subset_sums(a[:len(a)//2]))
right = sorted(subset_sums(a[len(a)//2:]))

```

Use: $n \leq 40$ where 2^{40} is hopeless but 2^{20} is instant.

Interactive Problems

```

import sys

print("?", x, flush=True)      # flush or you deadlock
resp = int(input())

print("!", answer, flush=True)

```

Careful: Do not use buffered output here, and do not rebind `input` to `sys.stdin.readline` without care — read exactly one line per query.

Stress Testing

```

import random, subprocess

for test in range(1000):
    n = random.randint(1, 8)          # keep it SMALL
    a = [random.randint(-5, 5) for _ in range(n)]
    data = f"{n}\n{' '.join(map(str, a))}\n"

    r1 = subprocess.run(["python", "fast.py"],
                        input=data, capture_output=True,
                        text=True).stdout
    r2 = subprocess.run(["python", "brute.py"],
                        input=data, capture_output=True,
                        text=True).stdout
    if r1.strip() != r2.strip():
        print("Mismatch:", data, r1, r2)
        break

```

Tip: Generate tiny tests. A counterexample you can read by hand beats a 200-line one you cannot.

Speeding Up CPython

- Put the work inside a function — locals beat globals.
- Bind hot attributes to locals: `ap = res.append` before the loop.
- Prefer comprehensions and `map` over explicit `append` loops.
- Use `sys.stdin.buffer.read()` for input and one `sys.stdout.write` for output.
- Replace inner loops with `sum`, `min`, `accumulate`, or slice assignment.
- Store grids as strings, and flags as `bytearray`.
- Submit on PyPy when the judge offers it.

■ Greedy: Proving It Works

- **Exchange argument** — show any optimal solution can be rewritten into the greedy one without getting worse.
- **Stays ahead** — show the greedy prefix is at least as good as any other at every step.
- If neither works, it is probably a DP.

Tip: Cheapest check of all: brute-force small cases and diff against your greedy.

■ Reading the Problem

- Is the answer guaranteed to exist, or must you print -1 ?
- How many test cases? Is the sum of n bounded?
- 0-based or 1-based indices in the statement?
- Can $n = 0$ or $n = 1$? Can the graph be disconnected?
- Are values negative?
- Is the graph a tree, a DAG, or fully general?

■ Bug Checklist

- `[[0]*m]*n` — shared rows, not a real 2D grid.
- Mutable default argument: `def f(a=[])`.
- State not reset between test cases (including `cache_clear()`).
- Off-by-one: inclusive vs half-open ranges.
- `/` where `//` was meant — floats break exactness.
- Forgetting `.rstrip()` on `sys.stdin.readline`.
- `a.pop(0)` or `s += ch` in a loop — silent $O(n^2)$.
- Recursion depth on 10^5 nodes.
- Reading `defaultdict[k]` and creating the key by accident.
- `float('inf')` contaminating integer arithmetic.

■ Time Management

- Read *every* problem first. The easiest is rarely problem A.
- Sort by solve count when the scoreboard shows it.
- Stuck 20 minutes with no idea? Switch and come back.
- Write the brute force when the statement is confusing — it clarifies the problem and doubles as a stress-test oracle.